Résumé et Exemples d'applications

La résolution d'un problème d'optimisation se fait généralement par étapes : à chaque étape on doit **faire un choix**. Par exemple, dans le problème du sac à dos, nous ajoutons les objets un par un, chaque ajout d'un objet constitue une étape : à chaque étape on doit **choisir** un objet à mettre dans le sac.

Le principe de la méthode **gloutonne** (*greedy* en anglais) est de, à chaque étape de la résolution du problème, faire le choix qui semble *le plus pertinent* sur le moment, avec l'espoir qu'au bout du compte, cela nous conduira vers une solution optimale du problème à résoudre. Autrement dit, on fait des **choix localement optimaux**, ce qui signifie de faire le choix qui semble le plus pertinent à chaque étape, dans l'espoir que ces choix mèneront à une solution globalement optimale.

En apparence, la solution la plus simple dans le cas du sac à dos serait d'écrire un algorithme qui teste toutes les combinaisons d'objets possibles et qui retient les solutions qui offrent un gain maximum.

Dans notre cas précis, avec seulement 4 objets, cette solution pourrait être envisagée, mais avec un plus grand nombre d'objets, le temps de calcul, même pour un ordinateur très puissant, deviendrait trop important. En effet l'algorithme qui testerait toutes les combinaisons possibles aurait une complexité en temps en $\mathbf{O(a^n)}$ avec a une constante et **n** le nombre d'objets.

On parle dans ce cas d'une **complexité exponentielle** (en temps), ce qu'on peut voir dans le graphique suivant (exécuté sur mon ordinateur personnel).

Comme on cherche à optimiser au maximum le gain, on va s'intéresser à la « valeur massique » de chaque objet.

I) Résolution et recherche par recuit simulé

Comme vu dans le cours le recuit simulé s'inspire de la thermodynamique dans le but de modéliser les systèmes physiques.

Objectif : Sortir d'un minimum local en acceptant avec une certaine probabilité une augmentation de la fonction objective (il s'agira de l'Energie E).

Avec, p la probabilité qu'un système physique passe d'un niveau d'énergie E1 à un niveau d'énergie E2.

L'algorithme de recuit simulé consiste en fait à simuler le processus de refroidissement très lent en partant d'une température assez élevée. L'algorithme permet de placer la variable X au voisinage du minimum absolu, mais pas de manière certaine. Avec un bon réglage de la vitesse de décroissance de la température, elle permet d'obtenir le minimum absolu avec une forte probabilité.

On commence par ***une valeur X choisie aléatoirement*** dans l'intervalle [a, b]. La suite des états est générée par la suite des étapes inscrites dans le code suivant :

****
Choisir une température de départ T=T0
Une solution initiale X0 ; k=0
Un paramètre de tolérance Beta appartenant à [0,1]

Tant qu'on peut abaisser la température
Tant que l'ensemble des voisins n'est pas vide
Générer une solution aléatoire dans le voisinage de la solution actuelle avec Xk+1 appartient à N(Xk)
Calculer la variation de f cad delta f= f(Xk+1)-f(Xk)

3) Si deltaf<=0 (le cout diminue il ya amélioration, et Xk+1 est accepté

Xk+1=Xk
K=k+1
Sinon (Si deltaf >=0 , le cout augmente,
on calcule la probabilité d'acceptation p=exp(-delta(f)/T),
On choisit une valeur beta appartenant à [0,1]
Si ***p>beta,*** on accepte la nouvelle solution malgré qu'elle n'améliore pas la fonction cout ;

Xk+1=Xk,
K=k+1

Sinon
La solution est rejetée

Fin Si
Fin Si

Répéter 1,2, et 3 jusqu'à l'équilibre thermodynamique
Fin de tant que (boucle 2)

Fin de tant que (boucle 1)
****

Exercice 1 : Algorithme de Recuit Simulé

En appliquant l'algorithme de recuit simulé, on y arrivé à une température T et à la solution courante s_0 tel que f(s_0)=100 ou f est la fonction objective du problème considéré. Cette solution a deux voisins s_1 et s_2 tel que f(s_1) = 102 et f(x_2)=99.

1) On suppose que T = 1000, calculer la probabilité de choisir un voisin si (i=1,2).

a) Dans le cas d'un problème de minimisation.
b) Dans le cas d'un problème de maximisation.

2) On prend T=5, calculer la probabilité de choisir un voisin s_i (i=1,2).
   a) Dans le cas d'un problème de minimisation.
   b) Dans le cas d'un problème de maximisation.
3) Préciser le comportement de l'algorithme de recuit simulé dans chacun des cas précédents 1 et 2.
4) On suppose que la version utilisée de recuit simulé est la version homogène sachant que le palier choisi est K=100 et que le refroidissement est géométrique avec taux de réduction égale à α=0.5.
5) Calculer le nombre d'appels de la fonction de voisinage entre T=1000 et T=5.

II) Résolution par algorithme de Tabou

Passons à la 2ème méthode de résolution, c'est dessous le pseudo-code d'une méthode de tabou:

- $s_{init} \leftarrow Glouton(Pb)$: // générer une solution initiale réalisable
- $s^* \leftarrow s_{init}$ // Mémoriser la meilleure solution
- $s \leftarrow s_{init}$ // solution courante
- $sTaboo \leftarrow \{s\}$ // solutions tabou
- $k \leftarrow 0$ // compteur itérations
- Répéter // Exploration des voisinages successifs
  1. $k \leftarrow k + 1$
  2. Choisir le meilleur voisin s' non tabou :
     - $s' \in Neighboor(s)$ tq $\forall s'' \in Neighboor(s), Obj(s') < Obj(s'')$ et $s' \notin sTaboo$
  3. Ajouter s' à $sTaboo$
  4. $s \leftarrow s'$
  5. si s' meilleur que s^* alors $s^* \leftarrow s'$
- Arrêt : $k \geq maxIter$ ou time out

Exercice 2 : Application de la recherche de Tabou pour le cas du problème de voyageur de commerce

*Ci-dessous, l'exemple de code tel que l'exemple étant le problème de **voyageur de commerce**, analyser cet algorithme et essayer de le traiter pour un cas d'utilisation différente tel que **le problème de sac à dos**, sur votre machine (sous python).*

```
from typing import List
import random
def voyageur_de_commerce (distances: List[List[int]], n: int, tabou_length: int, max_iter: int)
 - > List[int]:
   # Initialiser la liste tabou et la meilleure solution trouvée jusqu'à présent
   tabou_list = []
   best_solution = []
   best_distance = float("inf")
   # Initialise le compteur d'itérations
   iter_count = 0
     while True:
      # Générer une solution aléatoire
      solution = generate_random_solution(n)
       # Calculer la distance de la solution
      distance = calculate_distance(solution, distances)
          # Si la distance de la solution est meilleure que la meilleure solution trouvée jusqu'à présent,
      # mettre à jour la meilleure solution
      if distance < best_distance:
         best_solution = solution
         best_distance = distance
            # Ajouter la solution à la liste tabou
      tabou_list.append(solution)
          # Si la liste tabou a atteint sa longueur maximale, retirer le premier élément de la liste
      if len(tabou_list) > tabou_length:
```

```
            tabou_list.pop(0)
        # Incrémente le compteur d'itérations et vérifie si le nombre maximal d'itérations a été atteint
        iter_count += 1
        if iter_count >= max_iter:
            break
        return best_solution
    def generate_random_solution(n: int) -> List[int]:
    # Générer une solution aléatoire en mélangeant les villes
    cities = list(range(n))
    random.shuffle(cities)
    return cities
    def calculate_distance(solution: List[int], distances: List[List[int]]) -> int:
    # Calculer la distance totale de la solution en parcourant chaque ville
    distance = 0
    for i in range(len(solution) - 1):
        distance += distances[solution[i]][solution[i + 1]]
    return distance
# Créer la matrice de distances à partir des données fournies
distances = [
    [0, 12, 13, 15, 23, 25, 56, 27, 38, 29, 60, 32],
    [12, 0, 23, 22, 34, 56, 24, 78, 79, 34, 23, 24],
    [13, 23, 0, 25, 23, 19, 18, 42, 25, 23, 26, 33],
    [15, 22, 25, 0, 12, 24, 38, 27, 38, 49, 69, 29],
    [23, 34, 23, 12, 0, 54, 36, 71, 81, 19, 100, 22],
    [25, 56, 19, 24, 54, 0, 16, 27, 26, 37, 105, 72],
    [56, 24, 18, 38, 36, 16, 0, 18, 23, 24, 25, 26],
    [27, 78, 42, 27, 71, 27, 18, 0, 18, 19, 30, 53],
    [38, 79, 25, 38, 81, 26, 23, 18, 0, 29, 23, 22],
    [29, 34, 23, 49, 19, 37, 24, 19, 29, 0, 22, 42],
    [60, 23, 26, 69, 100, 105, 25, 30, 23, 22, 0, 36],
```

[32, 24, 33, 29, 22, 72, 26, 53, 22, 42, 36, 0]]

print(voyageur_de_commerce(distances, 12, 12, 1000))